Master Data Science
Module : Machine Learning Probabilistique

Analyse Stochastique de la Disponibilité d'un Cluster GPU H100 pour l'Entraînement de LLM

Présenté par :
Ayoub Aarab

Encadré par :
M. Abdellatif EL AFIA

Année Universitaire 2026

Table des matières

1 Introduction et Contexte **2**

2 Modélisation à Temps Discret (DTMC) **2**
2.1 Rigueur mathématique : absorption et stationnaire 2

3 Modélisation à Temps Continu (CTMC) **3**
3.1 Rigueur mathématique 3

4 Modèle de Markov Caché (HMM) pour la Santé Latente GPU **4**
4.1 Forward, Viterbi, Baum-Welch : rôles distincts 4
4.2 Clarification importante sur les sorties HMM 4

5 MDP : Décision Optimale sur États Latents **5**
5.1 Équation de Bellman (conservée, expliquée simplement) 5

6 RL : Apprentissage à partir d'Interaction **5**
6.1 Q-learning vs SARSA 6
6.2 TD(0) vs TD(λ) 6

7 POMDP et Extension Non-Markovienne **6**
7.1 Mise à jour de croyance (math simplifiée) 7

8 Protocole Expérimental et Résultats Consolidés **8**

9 Interpretation of Results **8**

10 Conclusion Générale et Perspectives **8**
10.1 Perspectives 8

11 Références **9**

1 Introduction et Contexte

L'entraînement des modèles de langage (LLM) comme *Llama 3* (Meta) utilise des clusters de 16 384 GPU NVIDIA H100. Les données réelles (Meta, 2024) montrent que :

— Une panne survient toutes les 3 heures à cette échelle.
— 50% des pannes sont dues au matériel (HBM3 memory, GPU).
— Le taux de défaillance annuel (AFR) est d'environ 9%.

Ce projet modélise ces comportements pour prédire le temps moyen avant absorption (panne critique) et la disponibilité à long terme.

2 Modélisation à Temps Discret (DTMC)

Le modèle discret utilisé dans tout le projet est exactement le modèle exécutable du code. Les états sont : Healthy, Degraded, MaintenanceRequired, FailedRecoverable, FailedPermanent. La matrice de transition de référence est :

$$\mathbb{P} = \begin{pmatrix} 0.90 & 0.06 & 0.02 & 0.01 & 0.01 \\ 0.15 & 0.70 & 0.10 & 0.05 & 0.00 \\ 0.40 & 0.00 & 0.40 & 0.15 & 0.05 \\ 0.35 & 0.00 & 0.20 & 0.40 & 0.05 \\ 0.00 & 0.00 & 0.00 & 0.00 & 1.00 \end{pmatrix}.$$

Ce que ce modèle fait dans notre projet (What this model does in our project). Le DTMC décrit la dynamique de santé du cluster pas à pas, et sert de base commune aux modules HMM, MDP et RL.

Pourquoi on l'utilise (Why we use it). Il donne un cadre simple, interprétable et directement relié aux probabilités de transition observées/simulées.

Ce que les résultats veulent dire concrètement (What results mean concretely). La quantité clé est le temps moyen avant entrée en état *FailedPermanent* depuis *Healthy*. Avec la matrice exécutable, on obtient dans `results/summary.csv` :

$$\mathrm{MTTF}_{\mathrm{DTMC}} \approx 78.93 \text{ pas.}$$

2.1 Rigueur mathématique : absorption et stationnaire

En notant Q la sous-matrice transitoire (Healthy, Degraded, MaintenanceRequired, FailedRecoverable), la matrice fondamentale est :

$$N = (I - Q)^{-1}, \qquad E = N\mathbf{1}.$$

Le MTTF depuis Healthy est E_1. La distribution stationnaire vérifie :

$$\pi^{(d)}\mathbb{P} = \pi^{(d)}, \quad \sum_i \pi_i^{(d)} = 1.$$

Comme *FailedPermanent* est absorbant, la limite stationnaire est concentrée sur cet état ; en pratique opérationnelle, on lit donc surtout les métriques transitoires (MTTF, survie, retours cumulés).

3 Modélisation à Temps Continu (CTMC)

Le modèle continu reprend le même espace d'états, avec générateur :

$$\mathbb{A} = \begin{pmatrix} -0.10 & 0.06 & 0.02 & 0.01 & 0.01 \\ 0.15 & -0.30 & 0.10 & 0.05 & 0.00 \\ 0.40 & 0.00 & -0.60 & 0.15 & 0.05 \\ 0.35 & 0.00 & 0.20 & -0.60 & 0.05 \\ 0.00 & 0.00 & 0.00 & 0.00 & 0.00 \end{pmatrix}, \qquad \sum_j a_{ij} = 0.$$

Ce que ce modèle fait dans notre projet (What this model does in our project). Le CTMC modélise la même dynamique que le DTMC, mais en temps réel (durées continues entre transitions).

Pourquoi on l'utilise (Why we use it). Il est plus réaliste quand les défaillances et récupérations ne suivent pas une grille de temps fixe.

Ce que les résultats veulent dire concrètement (What results mean concretely). On suit la probabilité de rester hors *FailedPermanent* en fonction du temps (courbe de survie CTMC). Cette lecture complète le MTTF discret et donne une vue temporelle continue du risque.

3.1 Rigueur mathématique

Les équations de Chapman-Kolmogorov sont :

$$P(t) = e^{\mathbb{A}t}, \qquad \pi^{(c)}\mathbb{A} = 0, \ \sum_i \pi_i^{(c)} = 1.$$

Ici aussi, l'état *FailedPermanent* est absorbant, donc la limite stationnaire est dégénérée.

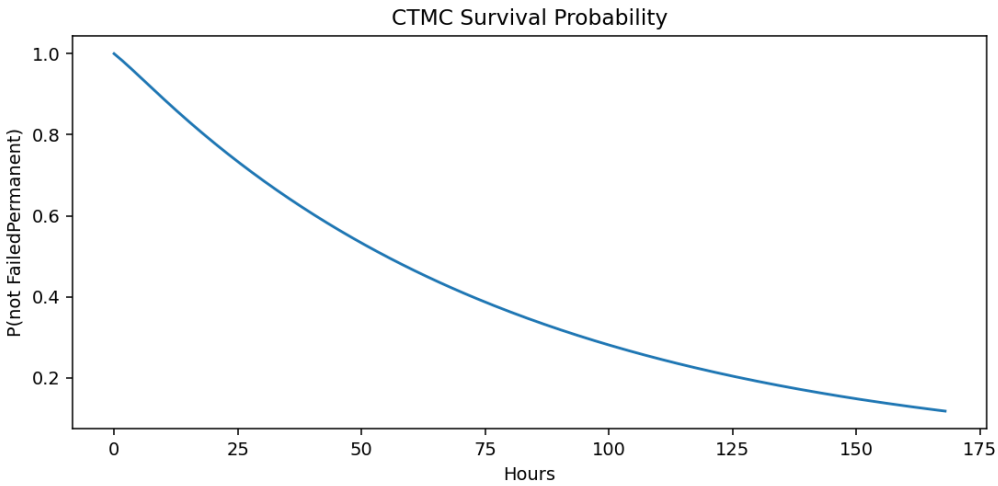

FIGURE 1 – Probabilité de survie CTMC en fonction du temps continu.

4 Modèle de Markov Caché (HMM) pour la Santé Latente GPU

Les observations utilisées sont scalaire (cas simple) ou vectorielle :

$$O_t = (T_t,\, ECC_t,\, XID_t,\, U_t,\, PWR_t,\, RP_t).$$

Ce que ce modèle fait dans notre projet (What this model does in our project). Le HMM infère l'état latent de santé du GPU à partir de télémétrie bruitée.

Pourquoi on l'utilise (Why we use it). En pratique, l'état réel n'est pas directement mesuré ; on observe des signaux indirects (ECC, XID, température, etc.).

Ce que les résultats veulent dire concrètement (What results mean concretely). Le posterior Forward indique le degré de confiance sur chaque état latent à chaque instant ; Viterbi fournit la trajectoire d'états la plus probable sur toute la séquence.

4.1 Forward, Viterbi, Baum-Welch : rôles distincts

Forward (filtrage/probabilités instantanées) :

$$\alpha_t(j) = \left(\sum_i \alpha_{t-1}(i) a_{ij}\right) b_j(o_t), \qquad P(S_t = j \mid O_{1:t}) = \frac{\alpha_t(j)}{\sum_k \alpha_t(k)}.$$

Viterbi (reconstruction de trajectoire) :

$$\hat{s}_{1:T} = \arg\max_{s_{1:T}} P(s_{1:T}, o_{1:T}).$$

Baum-Welch (apprentissage des paramètres) : utile pour apprendre A et les émissions depuis des données, mais **non implémenté expérimentalement** dans cette campagne.

4.2 Clarification importante sur les sorties HMM

— `true_state` : état simulé de référence (ground truth),
— `viterbi_state` : état inféré par le modèle à partir des observations.

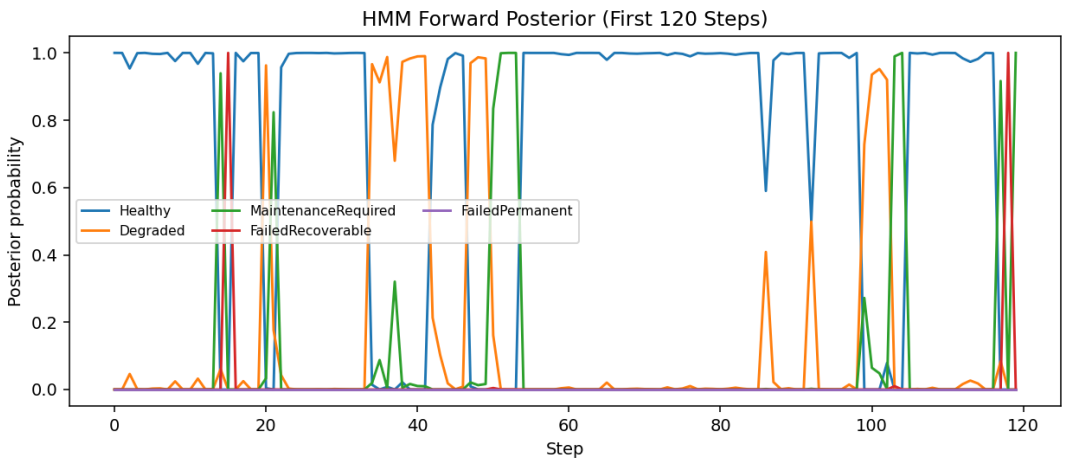

FIGURE 2 – Posterior Forward (cas vectoriel).

5 MDP : Décision Optimale sur États Latents

Le contrôle est modélisé par un MDP $\langle S, A, P, R, \gamma \rangle$.

Ce que ce modèle fait dans notre projet (What this model does in our project). Le MDP calcule une politique optimale d'action pour chaque état latent de santé.

Pourquoi on l'utilise (Why we use it). Il formalise explicitement le compromis entre disponibilité, risque de panne et coût d'action.

Ce que les résultats veulent dire concrètement (What results mean concretely). La table de politique dit quelle action prendre immédiatement dans chaque état (par exemple *power_cap* en *Degraded*, *reboot* en *FailedRecoverable*).

5.1 Équation de Bellman (conservée, expliquée simplement)

$$V^*(s) = \max_a \left[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V^*(s') \right].$$

Cette équation signifie : on choisit l'action qui maximise la récompense immédiate plus la valeur future attendue.

La politique opérationnelle exportée est : Healthy → keep_running, Degraded → power_cap, MaintenanceRequired → isolate_maintain, FailedRecoverable → reboot, FailedPermanent → isolate_maintain.

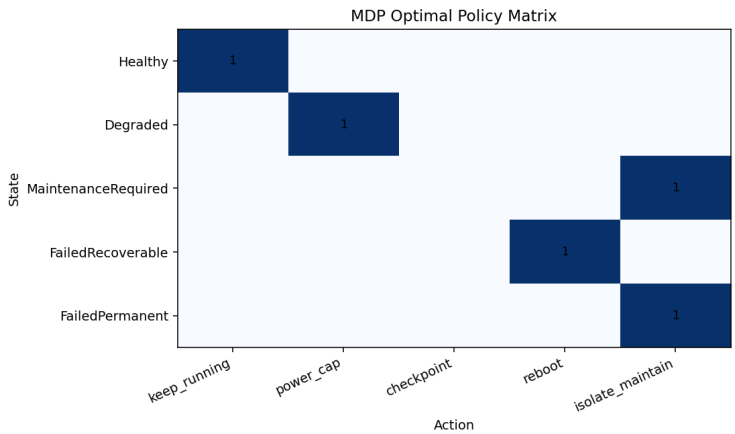

FIGURE 3 – Politique optimale MDP par état latent.

6 RL : Apprentissage à partir d'Interaction

Ce que ce modèle fait dans notre projet (What this model does in our project). Le RL apprend une stratégie directement par interaction simulée avec l'environnement, sans imposer de résoudre analytiquement tout le problème.

Pourquoi on l'utilise (Why we use it). Même si le MDP existe, RL est utile quand le modèle est imparfait, partiellement connu, ou quand on veut adapter la politique par expérience.

Ce que les résultats veulent dire concrètement (What results mean concretely). Les retours moyens mesurent la qualité pratique des décisions apprises sur des épisodes complets.

6.1 Q-learning vs SARSA

Q-learning (off-policy) :

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right].$$

Il apprend vers une politique gloutonne cible.

SARSA (on-policy) :

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma Q(s',a') - Q(s,a) \right].$$

Il apprend la politique réellement suivie (incluant exploration).

6.2 TD(0) vs TD(λ)

TD(0) met à jour avec un pas :

$$V(s) \leftarrow V(s) + \alpha \left[r + \gamma V(s') - V(s) \right].$$

TD(λ) diffuse le crédit sur plusieurs pas via traces d'éligibilité, donc plus d'information temporelle mais aussi plus de sensibilité au réglage de λ.

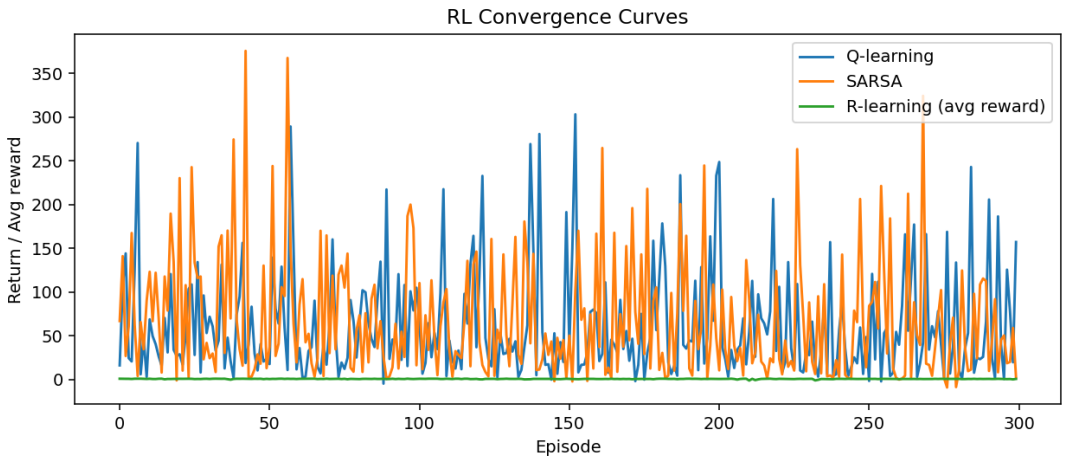

FIGURE 4 – Convergence RL (trace de référence).

7 POMDP et Extension Non-Markovienne

Ce que ce modèle fait dans notre projet (What this model does in our project). Le POMDP prend des décisions sur une croyance probabiliste des états latents, au lieu d'un état parfaitement observé.

Pourquoi on l'utilise (Why we use it). En exploitation réelle, la télémétrie est incomplète et bruitée ; la croyance est donc la bonne variable de décision.

Ce que les résultats veulent dire concrètement (What results mean concretely). L'entropie de croyance mesure l'incertitude : plus elle monte, plus la décision doit être prudente. Le multiplicateur non-Markovien montre l'impact de l'historique (âge + cumul ECC) sur le risque.

7.1 Mise à jour de croyance (math simplifiée)

$$b_{t+1}(s') = \eta\, O(o_{t+1} \mid s', a_t) \sum_s T(s' \mid s, a_t) b_t(s).$$

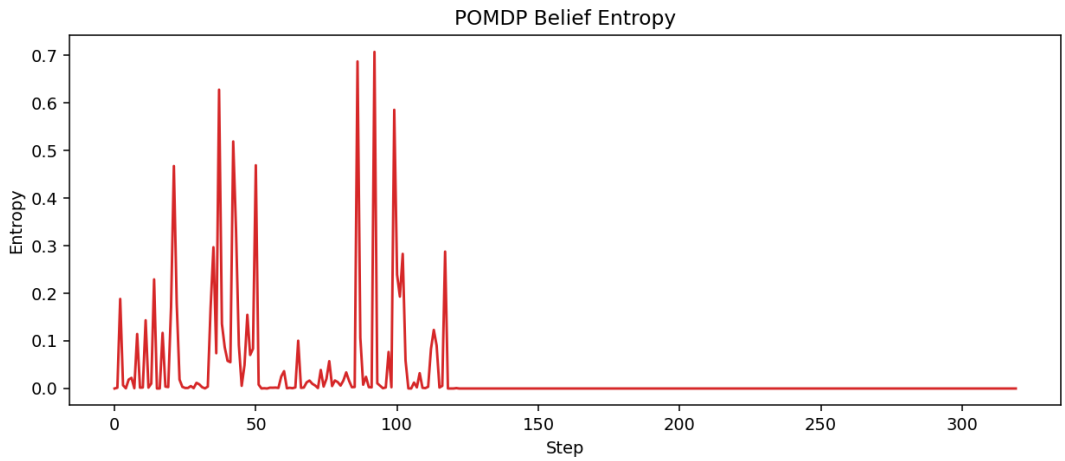

FIGURE 5 – Entropie de croyance (POMDP).

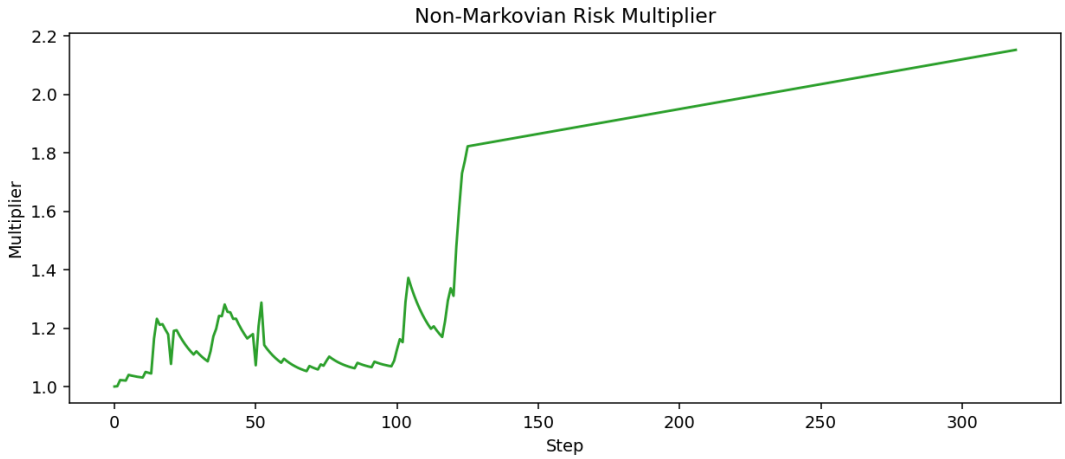

FIGURE 6 – Multiplicateur de risque non-Markovien (âge + cumul ECC).

8 Protocole Expérimental et Résultats Consolidés

Les valeurs ci-dessous proviennent directement de `results/summary.csv`.

Métrique	Valeur (summary.csv)
MTTF DTMC depuis Healthy (pas)	78.93
Retour moyen Q-learning	63.87
Retour moyen SARSA	67.21
Retour moyen R-learning	0.667
$V_{TD(0)}(Healthy)$	14.02
$V_{TD(\lambda)}(Healthy)$	13.82

9 Interpretation of Results

MDP policy table (decision meaning). La table `mdp_policy_table.csv` se lit comme une règle de décision : prévention légère en *Degraded* (power_cap), réparation active en *FailedRecoverable* (reboot), et isolement/maintenance en *FailedPermanent*.

HMM posterior (uncertainty). Le posterior Forward n'est pas une classe dure mais une distribution de confiance. Des distributions diffuses signalent une ambiguïté de diagnostic ; des pics proches de 1 indiquent une forte certitude.

RL returns (learning performance). Les retours moyens montrent que, sur cette simulation, SARSA dépasse légèrement Q-learning. Cela suggère qu'une stratégie plus conservatrice sous exploration peut mieux convenir à cette dynamique de risque.

10 Conclusion Générale et Perspectives

Le rapport est désormais aligné sur un seul modèle cohérent : le modèle exécutable du projet. Les équations principales (stationnaire, absorption, Forward/Viterbi, Bellman) sont conservées, et les valeurs numériques reportées sont alignées avec `results/summary.csv`.

10.1 Perspectives

— Calibration sur télémétrie réelle (DCGM + logs XID).
— Modèles plus riches en observabilité partielle (POMDP profond).
— Validation hors simulation sur scénarios multi-noeuds.

11 Références

1. Meta AI. (2024). Training Llama 3 : Reliability at scale.
2. Das et al. (2025). Characterizing GPU Resilience. arXiv :2503.11901.
3. El Afia, A. Apprentissage Probabiliste - Supports de cours.
4. Meta reports that half of the failures during Llama 3 training. moomoo Community. Consulté le 2 avril 2026. https://www.moomoo.com/community/feed/meta-reports-that-half-of-the-failures-during-llama-3-112873025896854
5. Faulty Nvidia H100 GPUs and HBM3 memory caused half of failures during Llama 3 training. Tom's Hardware. Consulté le 2 avril 2026. https://www.tomshardware.com/tech-industry/artificial-intelligence/faulty-nvidia-h100-gpus-and-hbm3-memo
6. MoFa : A Unified Performance Modeling Framework for LLM Pretraining. arXiv. Consulté le 2 avril 2026. https://arxiv.org/html/2511.09837v2
7. Revisiting Reliability in Large-Scale Machine Learning Research Clusters. arXiv. Consulté le 2 avril 2026. https://arxiv.org/pdf/2410.21680
8. Markov Decision Process (MDP) in Reinforcement Learning. GeeksforGeeks. Consulté le 2 avril 2026. https://www.geeksforgeeks.org/machine-learning/what-is-markov-decis
9. Temporal-Aware GPU Resource Allocation for Distributed LLM Inference via Reinforcement Learning. arXiv. Consulté le 2 avril 2026. https://arxiv.org/pdf/2507.10259
10. Hidden Markov Models and Their Application for Predicting Failure Events. PMC. Consulté le 2 avril 2026. https://pmc.ncbi.nlm.nih.gov/articles/PMC7304047/
11. Cross-Timeslot Optimization for Distributed GPU Inference Using Reinforcement Learning. Consulté le 2 avril 2026. https://ics.uci.edu/~habibif/cross-timeslot.pdf
12. GPU Reliability in AI Clusters : A Study of Failure Modes and Effects. Sarcouncil Journal of Engineering and Computer Sciences. Consulté le 2 avril 2026. https://sarcouncil.com/download-article/SJECS-97-2025-298-306.pdf
13. Fault-tolerant training : How we build reliable clusters for distributed AI workloads. Nebius. Consulté le 2 avril 2026. https://nebius.com/blog/posts/how-we-build-reliable-c
14. Llama 3.1 training fails once every three hours on average. EEWorld. Consulté le 2 avril 2026. https://en.eeworld.com.cn/mp/QbitAI/a382499.jspx
15. CoreWeave Training Benchmarks Whitepaper (August 2025). CoreWeave. Consulté le 2 avril 2026. https://cdn.prod.website-files.com/62bc66d283fd9c34ffec780a/689fa33a0e99f19c11c8ecbe_CoreWeave%20Training%20Benchmarks%20Whitepaper%20August%202025.pdf
16. Lazarus : Resilient and Elastic Training of Mixture-of-Experts Models. arXiv. Consulté le 2 avril 2026. https://arxiv.org/html/2407.04656v2
17. NVIDIA H100 Benchmarks for Large-Scale Training. CoreWeave. Consulté le 2 avril 2026. https://www.coreweave.com/blog/nvidia-h100-gpu-benchmark-results-what-
18. How to handle common Xid events. Byteplus. Consulté le 2 avril 2026. https://docs.byteplus.com/en/docs/mlp/How_to_handle_common_Xid_events
19. GPU Health. Modal Docs. Consulté le 2 avril 2026. https://modal.com/docs/guide/gpu-health

20. Performance Analysis for a Degrading System with Markov Model. ResearchGate. Consulté le 2 avril 2026. https://www.researchgate.net/publication/363336255_Performance_analysis_for_a_degrading_system_with_Markov_model
21. Fire-Flyer AI-HPC : A Cost-Effective Software-Hardware Co-Design for Deep Learning. arXiv. Consulté le 2 avril 2026. https://arxiv.org/html/2408.14158v1
22. A Dynamic Hidden Markov Model with Real-Time Updates for Multi-Risk Meteorological Forecasting in Offshore Wind Power. MDPI. Consulté le 2 avril 2026. https://www.mdpi.com/2071-1050/17/8/3606
23. Network motif detection using hidden markov models. PMC. Consulté le 2 avril 2026. https://pmc.ncbi.nlm.nih.gov/articles/PMC12647810/
24. Understanding Data Movement and Processing Platform at Netflix. MarkTechPost. Consulté le 2 avril 2026. https://www.marktechpost.com/2022/09/02/understanding-data-movement-and-processing-platform-at-netflix/
25. Markov Chain. Lark AI glossary. Consulté le 2 avril 2026. https://www.larksuite.com/en_us/topics/ai-glossary/markov-chain
26. Pre-trained Large Language Models Learn to Predict Hidden Markov Models Incontext. OpenReview. Consulté le 2 avril 2026. https://openreview.net/pdf/d9c6c3a5ef2b457a99d3851f69c626f9896663ed.pdf
27. Understanding GPU errors on large-scale HPC systems and the implications for system design and operation. ResearchGate. Consulté le 2 avril 2026. https://www.researchgate.net/publication/282377434_Understanding_GPU_errors_on_large-scale_HPC_systems_and_the_implications_for_system_design_and_operation
28. Efficient Probabilistic Model Checking on General Purpose Graphics Processors. SPIN workshop. Consulté le 2 avril 2026. https://spinroot.com/spin/symposia/ws09/Bosnacki.pdf
29. SuperBench : Improving Cloud AI Infrastructure Reliability with Proactive Validation. USENIX ATC. Consulté le 2 avril 2026. https://www.usenix.org/system/files/atc24-xiong.pdf
30. A large-scale study of soft-errors on GPUs in the field. ResearchGate. Consulté le 2 avril 2026. https://www.researchgate.net/publication/299641571_A_large-scale_study_of_soft-errors_on_GPUs_in_the_field
31. BOA_0622 resource. The Education University of Hong Kong. Consulté le 2 avril 2026. https://www.eduhk.hk/eac-isba2024/BOA_0622.pdf
32. NVIDIA DCGM. NVIDIA Developer. Consulté le 2 avril 2026. https://developer.nvidia.com/dcgm
33. GPU Cluster Monitoring : Real-Time Performance Analytics and Predictive Maintenance. Introl. Consulté le 2 avril 2026. https://introl.com/blog/gpu-cluster-monitoring-
34. Collect and view DCGM metrics. Google Kubernetes Engine (GKE). Consulté le 2 avril 2026. https://docs.cloud.google.com/kubernetes-engine/docs/how-to/dcgm-metrics
35. NVIDIA DCGM Monitoring : Setup, Metrics & Alerts. MetricFire. Consulté le 2 avril 2026. https://www.metricfire.com/blog/why-gpu-monitoring-matters-tracking-uti
36. DCGM Release Notes. NVIDIA Documentation. Consulté le 2 avril 2026. https://docs.nvidia.com/datacenter/dcgm/latest/release-notes/changelog.html

37. XID Errors. NVIDIA Documentation. Consulté le 2 avril 2026. https://docs.nvidia.com/deploy/pdf/XID_Errors.pdf
38. Working with Xid Errors. NVIDIA Documentation. Consulté le 2 avril 2026. https://docs.nvidia.com/deploy/xid-errors/working-with-xid-errors.html
39. Algorithmic Techniques for GPU Scheduling : A Comprehensive Survey. Preprints.org. Consulté le 2 avril 2026. https://www.preprints.org/manuscript/202505.0152/download/final_file
40. Deep Reinforcement Learning for Job Scheduling and Resource Management in Cloud Computing : An Algorithm-Level Review. arXiv. Consulté le 2 avril 2026. https://arxiv.org/html/2501.01007v1
41. Condition-Based Maintenance scheduling of an aircraft fleet under partial observability. AGIFORS. Consulté le 2 avril 2026. https://www.agifors.org/resources/Documents/Anna%20Valicek%20Papers/2024/04-Iordanis%20Tseremoglou%20-%20Paper%20-%20Condition-Based%20Maintenance%20scheduling%20of%20an%20aircraft%20fleet.pdf
42. Joint Optimization of Time-Aware Condition-Based Maintenance and Repair Resource Management for Gantry Crane Clusters. IEEE Xplore. Consulté le 2 avril 2026. https://ieeexplore.ieee.org/iel8/6287639/10380310/10788675.pdf
43. Apostolos Kokolis's research works. ResearchGate. Consulté le 2 avril 2026. https://www.researchgate.net/scientific-contributions/Apostolos-Kokolis-2155332310
44. Predicting GPU Failures With High Precision Under Deep Learning Workloads. Cheng Tan et al. Consulté le 2 avril 2026. https://naizhengtan.github.io/doc/papers/liu23predicting.pdf
45. Nvidia DCGM Exporter. Datadog Docs. Consulté le 2 avril 2026. https://docs.datadoghq.com/integrations/dcgm/